

AYRIK MATEMATİK (MTM2622)

15.04.2021 - 6. DERS

İletişim Bilgileri

Arş. Gör. Dr. Fatih Aylıkcı

Kimya-Metalurji Fakültesi

Matematik Mühendisliği A237

02123834616 - faylikci@yildiz.edu.tr

Avesis: <https://avesis.yildiz.edu.tr/faylikci>

Konular

1. Sayılar Teorisi ve Şifreleme
2. Modüler Aritmetik, Bölünebilirlik, Asal sayılar ve EBOB
3. Kongrüans çözümleri ve uygulamaları
4. Boolean cebrine giriş , Boolean fonksiyonları
5. Boolean temsilleri, Devrelerin minimizasyonu
6. Rekürans bağıntıları ve uygulamaları
7. Graf (çizge) ve graf (çizge) modelleri, Graf terminolojisi ve Özel tipten graflar
8. Euler ve Hamilton Yolları, En Kısa Yol Problemleri
9. Planar Graflar
10. Graflarda Renklendirme
11. Ağaçlar ve uygulamaları
12. Temel Ağaçlar, Minimum Temel Ağaçlar
13. Minimum Temel Ağaçlar ile ilgili uygulamalar

Kaynak Kitaplar

1. Kenneth H. Rosen, Discrete Mathematics and Its Applications, 7th Edition McGraw-Hill Companies, Inc., 2012.
2. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, Pearson Addison Wesley, 2004.
3. Kevin Ferland, Discrete Mathematics: An Introduction to Proofs and Combinatorics, 1st Edition, Cengage Learning, 2008.
4. Richard Johnsonbaugh, Discrete Mathematics, 7th Edition, Pearson Education, 2013.

Değerlendirme

%30 1. Vize

%30 2. Vize (Sınav, Ödev veya Quiz)

%40 Final

Özyineleme (Rekürans) Bağıntısı

Uygulamaları

- Bir dizinin tümevarımsal tanımını kendilerinden sonra gelen terimleri belirlemek için bir kuraldır ve kendinden sonra gelen bir veya daha fazla başlangıç terimini belirtir. Ayrıca, ikinci türün kuralına da (tümevarımsal tanımın bir parçası olsun ya da olmasın) **özyineleme ilişkisi** denir ve bu dizi eğer özyineleme ilişkisini sağlıyorsa özyineleme ilişkisinin çözümü olarak tanımlanır.
- Sayma problemlerini çözmek için kullanılan bu ilişkileri inceleyeceğiz. Mesela, bir bakteriyel kolonideki bakteri sayısı her saatte iki katına çıksın. Eğer koloni 5 bakteri ile başlarsa, n saat sonunda bakteri sayısı ne kadar artar? Bu problemi çözmek için, n saat sonundaki bakteri sayısı a_n olsun diyelim. Bakteri sayısı her saat başı 2 katına çıktığı için n 'i pozitif tamsayı kabul edersek $a_n = 2a_{n-1}$ gibi bir ilişkiden söz edebiliriz. Bu özyineleme ilişkisi, ilk durumun $a_0 = 5$ olduğu durum için tüm negatif olmayan n tamsayıları için a_n i bulmamızı sağlar. a_n için, n 'nin negatif tamsayı olmaması koşuluyla $a_n = 5 \cdot 2^n$ formülünü buluruz.
- Bu bölümde özyineleme ilişkisi kullanılarak çözülebilen sayma problemlerini inceleyeceğiz.

Özyineleme İlişkileri ile Modelleme

- ▶ Özyineleme ilişkisi; bileşik faiz hesapları, bir adadaki tavşanların nüfus popülasyonunu hesaplama, Hanoi kulelerindeki hareketlere karar verme, belirli özelliklere sahip bit dizgileri sayma gibi çok çeşitli problemlerin çözümünde kullanabiliriz.
- ▶ Örnek 1 de bir adadaki tavşan popülasyonunun özyineleme ilişkisi kullanılarak hesaplanması gösterilmektedir.

- **Örnek 1: Tavşanlar ve Fibonacci Sayıları** Fibonacci olarak da bilinen Leonardo Pisano 13. yy da Liber Abaci adlı kitabında bu problemleri ortaya koymuştur. Bir çift tavşan (bir dişi, bir erkek olmak üzere) alınmış ve bir adaya bırakılmıştır. Tavşanlar 2 aylık olana kadar herhangi bir üreme görülmemiştir. Fakat 2 aydan sonra her ay her doğan tavşan, bir çift tavşan yavrulamışlar (Şekilde gösterildiği gibi) ve doğan tavşanlar da aynı şekilde yaşam döngülerine devam etmişlerdir. Buna göre hiçbir tavşanın ölmediği varsayılarak, n ay sonra adadaki tavşan çifti sayısının özyineleme ilişkisini bulunuz.

Reproducing pairs (at least two months old)	Young pairs (less than two months old)	Month	Reproducing pairs	Young pairs	Total pairs
		1	0	1	1
		2	0	1	1
		3	1	1	2
		4	1	2	3
		5	2	3	5
		6	3	5	8
					

- **Çözüm:** n ay sonra tavşan çifti sayısını; $n=1,2,3,\dots$ Fibonacci dizisinin terimleri olmak üzere; f_n ile gösterelim.

Burada tavşan nüfusu özyineleme ilişkisi kullanılarak çözülebilir. İlk ayın sonunda, adadaki tavşan çifti sayısı $f_1 = 1$. Ayrıca bu tavşan çifti ilk ay çoğalamadıkları için $f_2 = 1$ olur. n ay sonundaki tavşan çifti nüfusunu bulmak için, güncel nüfusa, geçen ayın nüfusu yani f_{n-1} i, ve her yeni doğan çift 2 aylık olan bir diğer çiftten doğduğu için f_{n-2} ye eşit olacak olan yeni doğan çift sayısı eklenir.

Sonuç olarak, f_n dizisi için özyineleme ilişkisi

$$n \geq 3, f_1 = 1 \text{ ve } f_2 = 1$$

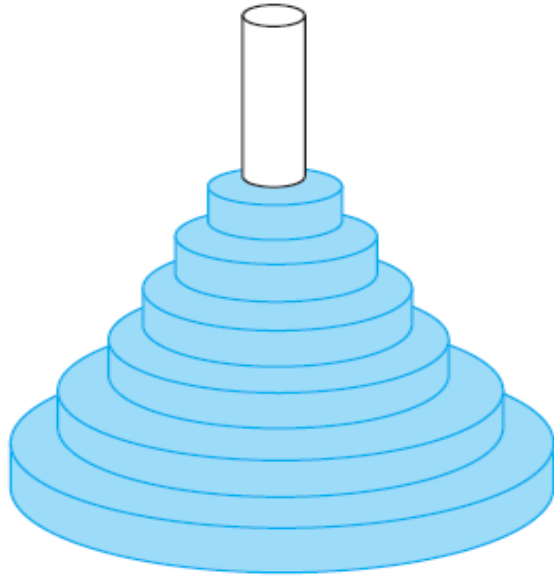
olmak üzere;

$$f_n = f_{n-1} + f_{n-2}$$

şeklinde bulunur. Bu özyineleme ilişkisi ve ilk durumlar gözönüne alındığında n ay sonra adadaki tavşan çifti sayısının n 'inci Fibonacci sayısı olduğunu görürüz.

- Örnek 2 de ise çok ünlü bir bulmacadan bahsediliyor.
- **Örnek 2: Hanoi Kuleleri** Popüler bir bulmaca olan 19. yy ın sonlarında Fransız matematikçi Edouard Lucas tarafından icat edilen Hanoi kuleleri; bir pano üzerine monte edilmiş 3 direk ve değişik boyutlarda disklerden oluşmaktadır. İlk önce diskler tek çubuk üzerinde en büyük disk en aşağıda olacak şekilde durur. (Şekilde gösterildiği gibi) Her hamlede yalnızca 1 disk taşınabilir ve hiçbir disk kendisinden küçük diskin üzerine konulamaz. Bulmacanın amacı en büyük disk aşağıda olacak şekilde 2. direk üzerine dizmektir.

H_n n tane dizilmiş diski olan Hanoi kulelerini çözmek için gerekli olan hamle sayısı olsun. $\{H_n\}$ dizisi için özyineleme ilişkisini bulunuz.



Peg 1



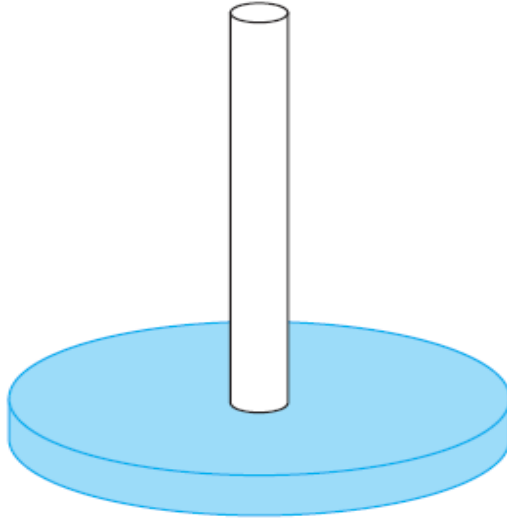
Peg 2



Peg 3

- **Çözüm:** 1. direkte n tane disk ile başlayalım. Bulmacanın kurallarını göz önünde bulundurarak H_{n-1} hamlede üstteki (n-1) tane disk 3. direğe dizebiliriz (Şekilde disklerin durumu görülmektedir). Bu hamleleri gerçekleştirirken en büyük disk 1. konumunda sabit tuttuk. Daha sonraki hamleyi en büyük disk 2. direğe geçirmek için yapıyoruz. En büyük disk her zaman 2. direkte sabit tutularak n-1 tane disk 3. direkten 2. ye geçirmek H_{n-1} tane daha hamle yapıyoruz. Burada da görülebileceği gibi bulmacanın birkaç hamlede çözülmesi mümkün değil. Çözümü şu şekilde gösterelim:

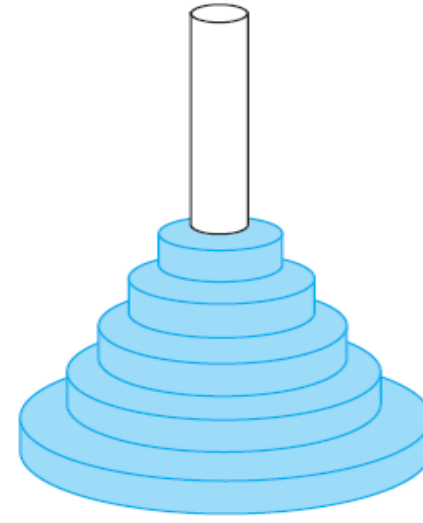
$$H_n = 2H_{n-1} + 1$$



Peg 1



Peg 2



Peg 3

- İlk durum $H_1 = 1$ olur. Çünkü bulmacanın kurallarına göre, bir disk 1. direktten diğerine yalnızca tek hamle kullanılarak geçirilebilir.

Bu özyineleme ilişkisini çözmek için yinelemeli yaklaşım kullanabiliriz. Öyle ki

$$\begin{aligned}H_n &= 2H_{n-1} + 1 \\&= 2(2H_{n-2} + 1) + 1 = 2^2 H_{n-2} + 2 + 1 \\&= 2^2 (2H_{n-3} + 1) + 2 + 1 = 2^3 H_{n-3} + 2^2 + 2 + 1 \\&\vdots \\&= 2^{n-1} H_1 + 2^{n-2} + 2^{n-3} + \dots + 2 + 1 \\&= 2^{n-1} + 2^{n-2} + 2^{n-3} + \dots + 2 + 1 \\&= 2^n - 1\end{aligned}$$

- H_n yi dizideki önceki terimler cinsinden göstermek için özyineleme ilişkisini tekrar tekrar kullandık. Sonraki de ise son eşitlik için $H_1 = 1$ kullandık. Son eşitlik bir geometrik serinin terimleri toplamını hesaplamak için kullanılan formüle dayanmaktadır.
- Yinelemeli yaklaşım $H_1 = 1$ ilk durumu ve $H_n = 2H_{n-1} + 1$ özyineleme ilişkisi için sonucu bulmamızı sağladı. Bu formül **tümevarım yöntemi** kullanılarak ispat edilebilir.
- Hanoi kuleleri ile ilgili bir efsanede bir grup keşişin 64 tane altın disk ile bulmacayı çözmeye çalıştıklarını söyler. Efsaneye göre keşişler bulmacayı bitirdiklerinde dünya yok olacaktır. Eğer her hamleyi 1 saniyede yapıyor olsalardı Keşişler bulmacaya başladıktan ne kadar sonra dünya yok olurdu.

Yukarıda bulduğunuz formüle göre keşişlerin bulmacayı bitirebilmeleri için

$$2^{64} - 1 = 18.446.744.073.709.551.615$$

tane hamleye ihtiyaçları olur. Her hamleyi de 1 saniyede yaptıklarını düşünecek olursak 500 milyar yıl sonra ancak bulmacayı tamamlarlar. Bu da demek oluyor ki dünya şimdiye kadar geçen zamandan çok daha fazla hayatta kalacak.

- **Uyarı:** Hanoi kulelerinin en eski ve en ilginç türlerinden biri, 1907 yılında Henry Dudeney tarafından The Canterbury Puzzles adlı kitabında yayınlanan Reve bulmacasıdır. Reve bir seyyaha 4 tabureden birinin üzerinde bulunan değişik büyüklükteki peynir kalıplarını bir diğer tabureye küçük peynir üzerine kendisinden büyük peynir gelmeyecek şekilde hepsini taşıması için meydan okur. Reve bulmacasını direkler ve diskler kullanılmış şekilde düşünelim. Hanoi kuleleriyle aynı kurallara sahiptir. Tek farkı 3 yerine 4 direk kullanılır. Size şaşırtıcı gelebilecek yanı ise şimdiye kadar n disk ile olan Reve bulmacasını çözmek için gerekli olan minimum hamle sayısı bulunamamıştır. Buna rağmen, günümüzde problemi çözmek için gereken hamle sayısının Frame ve Stewart tarafından 1939 yılında icat edilen algoritmada kullanılan hamle sayısına eşit olduğu gibi bir varsayım vardır.

- ▶ 3. örnek özyineleme ilişkilerinin belirli bir özelliği olan istenilen uzunluktaki bit dizgilerinin sayılması için nasıl kullanılabileceğini göstermektedir.
- ▶ **Örnek 3:** n uzunluğunda ardışık iki tane 0 içermeyen bit dizgileri için; ilk durumu ve özyineleme ilişkisini bulunuz. Bu şekilde uzunluğu 5 olan kaç tane bit dizgisi vardır?
- ▶ **Çözüm:** Uzunluğu n olan ve ardışık iki tane 0 içermeyen bit dizgilerinin sayısı a_n olsun. $\{a_n\}$ için bir özyineleme ilişkisi bulmak için, toplam kuralını kullanarak, ardışık 2 sıfır içermeyen uzunluğu n olan bit dizgileri sayısı; kendisi gibi ardışık 2 sıfır içermeyen 0 ile biten ve 1 ile biten bit dizgileri sayısına eşittir diyebiliriz. Bit dizgisinde en az 3 bit olması için $n \geq 3$ olduğunu kabul edeceğiz.
- ▶ 1 ile biten ve ardışık 2 tane 0 içermeyen n uzunluğundaki bit dizgisi, aynı zamanda $n-1$ uzunluğunda ardışık 2 tane 0 içermeyen $n-1$ uzunluğunda bir bit dizgisidir. Sonuç olarak a_{n-1} tane bit dizgisi vardır.
- ▶ İki ardışık 0 içermeyen, 0 ile biten n uzunluğundaki bir bit dizgisinde $(n-1)$ tane bit vardır. Aksi halde 2 ardışık 0 ile bitmesi gerekir. n uzunluğunda sonu 0 ile biten ve ardışık iki tane 0 içermeyen bir bit dizgisi aynı zamanda, iki tane ardışık 0 içermeyen ve 10 ile biten $n-2$ uzunluğunda bir bit dizgisidir. Buna bağlı olarak, bu şekilde a_{n-2} adet bit dizgisi vardır.

- Şekilde gösterildiği gibi özetleyecek olursak, $n \geq 3$ için

$$a_n = a_{n-1} + a_{n-2}$$

olur.

İlk durumlar için $a_1 = 2$ olur. Çünkü 1 uzunluğundaki her iki dizi, 0 ve 1, ardışık iki 0 içermez. Ayrıca $a_2 = 3$ diyebiliriz. Çünkü uzunluğu 2 olan bit dizgileri 01, 10 ve 11 olur. a_5 i elde etmek için, 3 kere özyineleme ilişkisi kullanmamız gerekir.

$$a_3 = a_2 + a_1 = 3 + 2 = 5$$

$$a_4 = a_3 + a_2 = 5 + 3 = 8$$

$$a_5 = a_4 + a_3 = 8 + 5 = 13$$

n uzunluğundaki ardışık
iki 0 içermeyen bit
dizgisi sayısı

1 ile biten	$n-1$ uzunluğundaki ardışık iki 0 içermeyen herhangi bir bit dizgisi
0 ile biten	$n-2$ uzunluğundaki ardışık iki 0 içermeyen herhangi bir bit dizgisi

1

a_{n-1}

10

a_{n-2}

Toplam:

$$a_n = a_{n-1} + a_{n-2}$$

- **Uyarı:** Fibonacci dizisindeki gibi aynı özyineleme ilişkisinin a_n i sağladığına dikkat ediniz. Çünkü; $a_1 = f_3$ ve $a_n = f_{n+2}$ şeklinde devam eder.
- Örnek 4'te özyineleme ilişkisi kullanılıp, belirli geçerlilik kontrolleri yapılarak kaç tane kelime oluşturulabileceği gösterilmiştir.
- **Örnek 4: Kelime Listelenmesi** Bir bilgisayar sistemi, çift sayıda 0 içeren onluk sistemde sayılarla yazılmış bir kelimeyi geçerli, bir başka deyişle doğru kabul eder. Mesela, 1230407869 geçerliyken 120987045608 geçerli değildir. n basamaklı bir kelime a_n olsun diyelim. a_n için özyineleme ilişkisini bulunuz.
- **Çözüm:** Burada 10 basamaklı bir dizgi olduğu ve sadece bir dizgi, yani 0 dizgisi geçerli olmadığına göre $a_1 = 9$ olduğuna dikkat ediniz. $n-1$ basamaklı dizilerden n basamaklı geçerli bit dizgilerini nasıl bulacağımıza karar vererek, bu bit dizgisi için bir özyineleme ilişkisi türetilebilir. n basamaklı bir diziden 1 basamak eksik geçerli bit dizgilerini oluşturmak için 2 yol vardır.
- Birincisi, n basamaklı geçerli bir bit dizgisi; $n-1$ basamaklı bir bit dizgisinin sonuna 0'dan başka bir bit eklenerek elde edilebilir. Ekleme işlemi 9 şekilde yapılır. Böylece, n basamaklı doğru ve geçerli bir bit dizgisi bu kurala göre $9a_{n-1}$ şekilde oluşturulabilir.

- İkincisi, n basamaklı geçerli bir bit dizgisi geçerli olmayan n-1 uzunluğunda bir bit dizgisinin sonuna 0 eklenerek elde edilebilir. (Bu çift sayıda 0 içeren bit dizgisi elde etmemizi sağlar. Çünkü n-1 uzunluğunda geçerli olmayan bit dizgisi tek sayıda 0 içermiştir.) Bunu sağlayan yolların sayısı sağlamayan n-1 bit dizgilerinin sayısına eşittir. Çünkü n-1 uzunluğunda 10^{n-1} tane bit dizgisi vardır ve bunların a_{n-1} tanesi geçerlidir. $10^{n-1} - a_{n-1}$ tane geçerli n basamaklı bit dizgisi, n-1 uzunluğundaki geçerli olmayan bit dizgisinin sonuna 0 eklenerek elde edilir.
- n uzunluğundaki tüm geçerli bit dizgileri bu 2 şekilden birinde oluşturulabildiğine göre:

$$\begin{aligned}a_n &= 9a_{n-1} + (10^{n-1} - a_{n-1}) \\ &= 8a_{n-1} + 10^{n-1}\end{aligned}$$

tane n uzunluğunda geçerli bit dizgisi vardır.

- Örnek 5'te oluşturulan özyineleme ilişkisi bir çok değişik durumda kullanılmaktadır.
- **Örnek 5:** $n+1$ tane sayının, $x_0.x_1.x_2...x_n$ çarpım sırasını belirlemek için kaç şekilde paranteze alınabileceğini C_n için özyineleme ilişkisi kurarak bulunuz. (C_n : kaç şekilde paranteze alınabileceği.)

Mesela, $C_3 = 5$ olur. Çünkü x_0, x_1, x_2, x_3 için çarpım sırasına karar vermek için 5 farklı paranteze alma şekli vardır.

$$\begin{array}{lll} ((x_0.x_1).x_2).x_3 & (x_0.(x_1.x_2)).x_3 & (x_0.x_1).(x_2.x_3) \\ x_0.((x_1.x_2).x_3) & x_0.(x_1.(x_2.x_3)) & \end{array}$$

- **Çözüm:** C_n için bir özyineleme ilişkisi oluşturmak için $x_0.x_1.x_2...x_n$ e koyduğumuz parantez nerede olursa olsun dışarıda bir «.» operatörü parantezin dışında kalır. Örneğin $(x_0.(x_1.x_2)).x_3$ de «.» son operatördür. $(x_0.x_1).(x_2.x_3)$ de ise ikinci operatör olduğu görülmektedir. Son operatör 2 tane $n+1$ sayının arasında bulunur, örneğin x_k ve x_{k+1} . Son operatörün x_k ve x_{k+1} arasında bulunduğu $n+1$ tane sayının çarpım sırası için parantezleri yerleştirmenin $C_k C_{n-k-1}$ tane yolu vardır. Çünkü $x_0.x_1.x_2...x_k$ çarpımına parantez eklemek için ve $k+1$ çarpılan sayının sırasını bulmak için C_k yol vardır ve $x_{k+1}.x_{k+2}.x_{k+3}...x_n$ çarpımına parantez eklemek için ve bu $n-k$ çarpılan sayının sırasını bulmak için de C_{n-k-1} yol vardır. Bu son operatör

$$\begin{aligned} C_n &= C_0 C_{n-1} + C_1 C_{n-2} + \dots + C_{n-2} C_1 + C_{n-1} C_0 \\ &= \sum_{k=0}^{n-1} C_k C_{n-k-1} \end{aligned}$$

olacak şekilde $n+1$ sayı arasından herhangi iki tanesinin arasında bulunabilir. İlk durumların $C_0 = 1$ ve $C_1 = 1$ olduğuna dikkat ediniz.

Örnek 5 deki özyineleme ilişkisi **üretici fonksiyonlarla** çözülebilir.

$$C_n = C(2n, n) / (n+1) \text{ ve } C_n \sim \frac{4^n}{n^{3/2} \sqrt{\pi}}$$

- $\{C_n\}$ dizisi **Katalan sayılarının dizisidir**. İsmi Eugene Charles Catalan'dan almıştır. Bu dizi bu bölümde gösterdiğimiz çözümlerin yanında bir çok sayma probleminin çözümünde kullanılır.

Algoritmalar ve Özyineleme İlişkileri (Bağıntıları)

- ▶ Özyineleme ilişkileri algoritma ve bunların karmaşıklığı çalışmalarında çok önemli bir rol oynar. Bu bölümü dinamik programlama olarak bilinen ve birçok en iyileme problemlerinde de etkin bir şekilde kullanılan algorithmadan bahsederek sonlandıracağız.
- ▶ Dinamik programlama anlayışını izleyen bir algoritma, problemi daha basit örtüşen alt problemlere ayırır ve sonucu alt problemlerin çözümlerini kullanarak bulur. Genelde, özyineleme ilişkileri alt problemlerin çözümlerinden tüm sonucu bulmak için kullanılır. Dinamik programlama Ekonomi, Bilgisayar Görüsü, Konuşma Tanıma, Yapay Zeka, Bilgisayar Grafiği, Biyoinformatik gibi çok farklı alanlardaki problemleri çözmek için kullanılır. Bu bölümde Zaman çizelgelemesi problemlerinin çözümlerinde algoritma oluşturup dinamik programlama ile çözmeyi göstereceğiz. Bunu yapmadan önce 1950'lerde matematikçi Richard Bellman tarafından ilk defa tanıtılan dinamik programlama adının nereden geldiğine bakacağız.

- Bellman ABD savunma bakanlığının matematiksel araştırmalara düşmanca bir tutum içerisinde olduğu zamanlarda ABD askeri projeleri yapan RAND firması için çalışmaktaydı. Bellman finansman sağlamak için tek ihtiyacı olan şeyin planlama ve zaman çizelgeleme problemlerini çözmek için geliştirdiği metoda içinde matematik geçmeyen bir isim vermek olduğuna karar verdi. Dinamik sıfatını kullanmaya karar verdi çünkü söylediğine göre «Dinamik kelimesini küçültücü aşağılayıcı anlamlarında kullanmak mümkün değildi» Ayrıca dinamik programlama «bir kongre üyesinin bile itiraz edemeyeceği bir şeydi.»

- **Bir Dinamik Programlama Örneği** Problemde amacımız tek bir amfi için mümkün olduğu kadar çok ders planlamak. Bu derslerde önceden ayarlanmış başlama ve bitiş tarihleri olup, bir kere ders başladığında bitene kadar devam etmektedir. Bir defada 2 ders aynı amfide yapılamaz. Bir dersin bitiş saati diğer bir dersin başlama saati sayılabilir. Şimdi amacımızın mümkün olan en iyi şekilde ders saatlerini planlamak değil ama planlanmış dersleri için mümkün olan en fazla birleşik katılımı bulmak olsun.

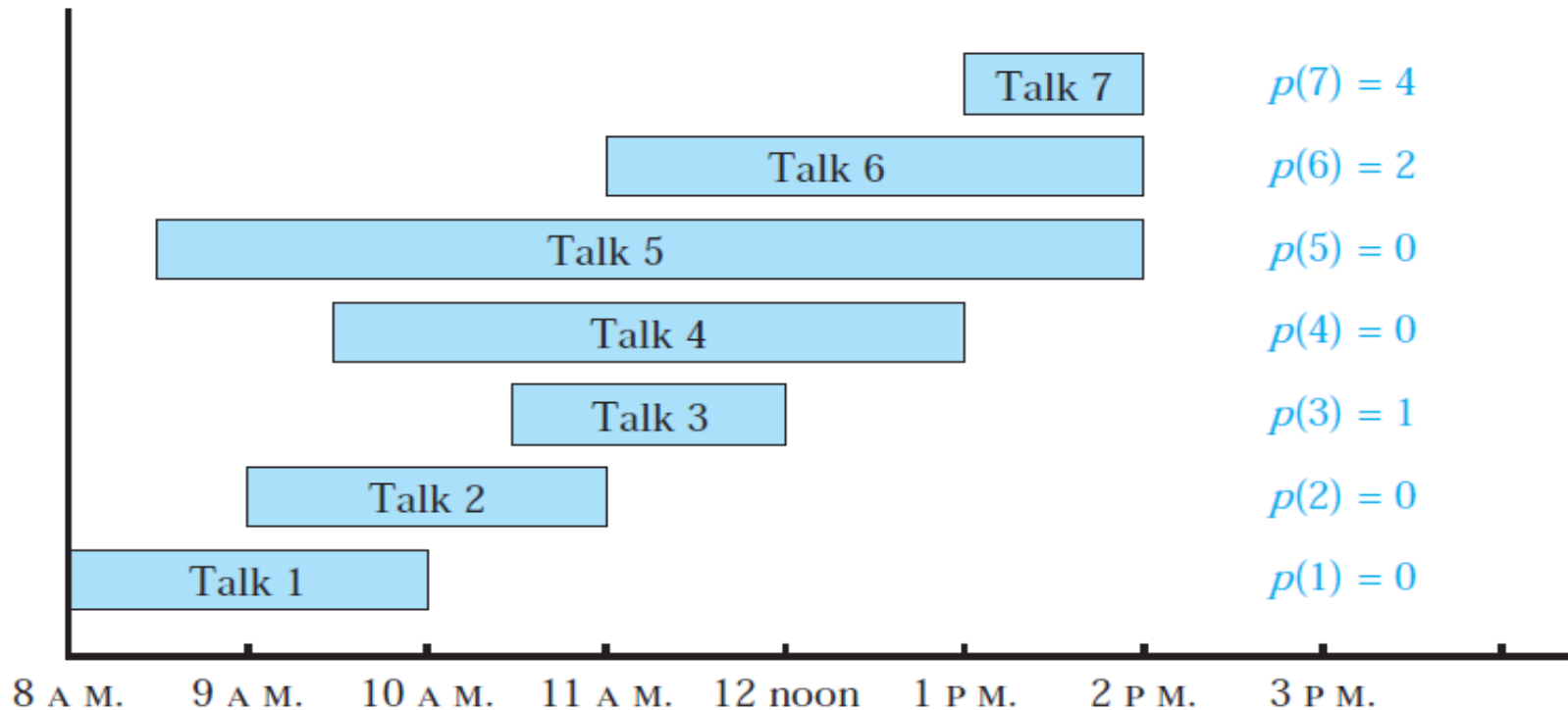
Bu problemi n tane ders olduğunu, j . dersin t_j de başladığını, e_j de bittiğini ve w_j tane öğrencinin katıldığını düşünerek formülize edelim. Katılan toplam öğrenci sayısının maksimum olduğu bir planlama istiyoruz. Tüm planlanmış konuşmalardaki w_j toplamını en üst seviyeye çıkartmak için derslerin alt kümesini planlamak istiyoruz. (Bir öğrenci birden fazla derse katıldığında, bu öğrencinin katıldığı derslere göre sayıldığına dikkat ediniz.) İlk j tane dersin en iyi planlaması için maksimum katılan sayısını $T(j)$ ile gösterelim. Öyleyse $T(j)$; n tane ders için maksimum katılacak katılımcı sayısıdır.

- Artan bitiş zamanlarına göre ilk önce dersleri sıraladık. Bunu yaptıktan sonra, dersleri yeniden $e_1 \leq e_2 \leq \dots \leq e_n$ olarak numaralandırdık. O halde 2 ders aynı planlamanın bir parçası ise ve herhangi bir çakışma yoksa uyumludur diyebiliriz. (Aynı zamanda biri bitip diğeri başlama ihtimalinden başka). $p(j)$ en büyük olmak üzere eğer böyle bir tamsayı varsa $e_i \leq s_j$ için i tamsayısı, $i < j$ olur. Aksi halde $p(j)=0$ diyebiliriz. Eğer böyle bir ders mevcut ise, j . dersten önce biten, ders j ile uyumlu olmak üzere $p(j)$, dersler arasında en geç biten derstir. Eğer böyle bir ders mevcut değil ise $p(j)=0$ olur.

- Örnek 6: Şekil 5’te gösterilmekte olan 7 ders için başlama ve bitiş saatlerini bulun.

Ders 1: başlama 8 bitiş 10	Ders 5: başlama 8:30 bitiş 14
Ders 2: başlama 9 bitiş 11	Ders 6: başlama 11 bitiş 14
Ders 3: başlama 10:30 bitiş 12	Ders 7: başlama 13 bitiş 14
Ders 4: başlama 9:30 bitiş 13	

$j=1,2,\dots,7$ için $p(j)$ yi bulunuz.



- **Çözüm:** $p(1)=0$ ve $p(2)=0$, çünkü her iki dersin başlamasından önce hiçbir ders bitmiyor. $p(3)=1$ olur çünkü ders 3 ve ders 1 uyumludur. Fakat ders 3 ve ders 2 uyumlu değildir. $p(4)=0$ çünkü ders 4 1,2 veya 3 numaralı derslerin hiçbirisiyle uyumlu değildir. $p(5)=0$ çünkü ders 5 1,2,3 veya 4 numaralı derslerin hiçbirisiyle uyumlu değildir. $p(6)=2$ çünkü ders 6 ders 2 ile uyumludur. Son olarak $p(7)=4$ 'tür. Çünkü ders 7 ve ders 4 uyumludur. Fakat ders 7 ne ders 5 ne de ders 6 ile uyumlu değildir.
- Bu problem için bir dinamik programlama algoritması geliştirmek için öncelikli olarak bir anahtar özyineleme ilişkisi oluşturuyoruz. Bunu yapmak için, eğer $j \leq n$ olursa, ilk j tane ders için en iyi planlama için 2 tane ihtimal vardır. (n tane dersin bitiş zamanına göre artarak sıralandığını farzettığımızı hatırlayınız.) 1. durum j en iyi planlamaya dahildir. 2. durum ise değildir.

- ▶ (i) durum: Diğer derslerin hiç birinin j ile uyumlu olmaması durumunda biliyoruz ki $p(j)+1, \dots, j-1$ bu planlamaya dahil değildir. Dahası bu en uygun planlama $1,2,\dots,p(j)$ derslerini de içermelidir. $1,2,\dots,p(j)$ dersleri için j dersi eklenerek daha iyi bir planlama olsaydı tüm en uygun planlardan daha iyi bir planımız olmuş olacaktı. Sonuç olarak (i) durumu için $T(j) = w_j + T(p(j))$
- ▶ (ii) durum: Ders j eğer en uygun plana sahip değilse, $1,2,\dots,j'$ den en uygun plan $1,2,\dots,j-1$ için olan en uygun planla aynı olur. Sonuç olarak, (ii) durumunda $T(j)=T(j-1)$ olur.
- ▶ Durum (i) ve (ii) yi birleştirdiğimiz takdirde özyineleme ilişkisini

$$T(j) = \max(w_j + T(p(j)), T(j-1))$$

olarak buluruz.

- Bu özyineleme ilişkisini bulduğumuza göre en uygun algoritmayı, Algoritma 1'i maksimum toplam katılımcı sayısını bulmak için oluşturmaya başlayabiliriz. Algoritma biz her hesapladığımızda $T(j)$ değerini saklamakta oldukça verimli bir algoritmadır. Bize $T(j)$ değerini bir kere hesaplama olanağı veriyor. Eğer bunu yapmamış olsaydık algoritmada üssel kötü durum karmaşıklığı olacaktı. Her hesaplanan değer saklanma sürecine not etme, saklama (memorization) diyoruz. Not etme, özyineli algoritmaları verimli yapmak için çok önemli bir tekniktir.

ALGORITHM 1 Dynamic Programming Algorithm for Scheduling Talks.

```
procedure Maximum Attendees ( $s_1, s_2, \dots, s_n$ : start times of talks;  
     $e_1, e_2, \dots, e_n$ : end times of talks;  $w_1, w_2, \dots, w_n$ : number of attendees to talks)  
    sort talks by end time and relabel so that  $e_1 \leq e_2 \leq \dots \leq e_n$   
    for  $j := 1$  to  $n$   
        if no job  $i$  with  $i < j$  is compatible with job  $j$   
             $p(j) = 0$   
        else  $p(j) := \max\{i \mid i < j \text{ and job } i \text{ is compatible with job } j\}$   
         $T(0) := 0$   
    for  $j := 1$  to  $n$   
         $T(j) := \max(w_j + T(p(j)), T(j - 1))$   
    return  $T(n)$  { $T(n)$  is the maximum number of attendees}
```

- Algoritma 1’de maksimum katılımcı sayısına ders planını bularak ulaşabiliriz. Fakat biz bu maksimumu sağlayan bir plan bulmuyoruz. Planlamamız gereken dersleri bulmak için, sadece ve sadece $w_j + T(p(j)) \geq T(j-1)$ olduğu durumda, ilk j ders için en iyi sonuca ait olan ders j yi kullanıyoruz.
- Algoritma 1, dinamik programlamanın j nin $1 \leq j \leq n-1$ olduğu, her biri ilk j dersin maksimum katılımını tespit eden alt problem çözümlerinin optimal sonuçlarını kullanarak maksimum katılımı bulmaya yönelik iyi bir örnektir.